

EV Smart Charging System

ML Models + LP Optimizer: Architecture & Coupling

April 2026 | Project Report

1. System Overview

The EV Smart Charging system is a microservice platform that schedules charging power across multiple electric vehicles simultaneously. It is built around a **Predict-Then-Optimize** architecture: Machine Learning models first produce predictions about each vehicle's future behaviour, and a Linear Programming (LP) optimizer then allocates power using those predictions as inputs to its constraint set.

The system runs **three strategies in parallel** for every simulation to quantify ML value: (1) **AI + ML** — LP optimizer with ML departure predictions and demand forecasts; (2) **AI only** — LP optimizer with historical mean departures, no ML; (3) **FCFS** — First-Come-First-Served greedy, no ML, no optimization.

1	ML Service	Predict departure per vehicle	Stay duration (q10/q50/q90)
2	ML Service	Forecast future arrivals (aggregate)	Count + kWh per future window
3	Gateway Orchestrator	Build optimizer request	Per-vehicle slots + future arrivals
4	LP Optimizer	Solve 2-phase LP	Power schedule per vehicle per slot
5	Gateway	Return results + collect feedback	Satisfaction %, energy delivered

Table 1 — End-to-end data flow through the system.

2. Machine Learning Models

2.1 Departure Time Predictor (M1)

A **Quantile Regression** model (HistGradientBoostingRegressor) trained on 23,444 real sessions from the ACN-Data Caltech dataset. For each connected vehicle it outputs three quantile estimates of how long the vehicle will stay:

Quantile	Meaning	Use in optimizer
q10 (10th percentile)	Earliest realistic departure	Conservative lower bound
q50 (median)	Most likely departure time	Default departure_slot input
q90 (90th percentile)	Latest realistic departure	Optimistic upper bound

Table 2 — Departure quantile outputs and their roles.

arrival_time	Timestamp of vehicle connection (OCPP event)
arrival_hour	Hour of day (0–23), captures commuter patterns
day_of_week	0=Monday ... 6=Sunday
is_weekend	Binary flag
requested_energy_kwh	Energy needed = battery_capacity × (target_pct – current_pct)
site_id / cluster_id	Charging site identifier (affects stay distribution)

Table 3 — Departure model input features.

Performance: test-set MAE $\approx$ 112 min vs. 184 min for the ACN per-hour historical mean baseline — a **~39% reduction in departure prediction error**.

2.2 Arrivals / Demand Forecaster (M2)

A time-series model that predicts how many EVs will arrive in each upcoming 30-minute window and how much energy (kWh) they will need in total. The optimizer uses this to **reserve grid capacity** for vehicles that have not yet arrived — preventing over-commitment to current vehicles.

3. LP Optimizer

The optimizer (lp_scheduler.py) uses **scipy HiGHS interior-point LP** to allocate charging power across all vehicles and time slots simultaneously. Time is discretized into 15-minute slots. Each decision variable x_{ik} is the power (kW) assigned to vehicle i in slot k .

3.1 Inputs to the Optimizer

vehicles[i].departure_slot	ML M1 (q50)	When vehicle i is expected to leave
vehicles[i].energy_needed_kwh	Battery state (OCPP)	kWh still needed to reach target SoC
vehicles[i].arrival_slot	OCPP event	Slot when vehicle connected
vehicles[i].max_charge_kw	Hardware spec	Maximum charger power limit
transformer_capacity_kw	Grid config	Total site power budget (kW)
base_load_per_slot_kw[]	Building metering	Non-EV background load per slot
predicted_future_arrivals[]	ML M2 forecast	Expected new EVs + kWh per future slot

Table 4 — Optimizer inputs and their sources.

4. Constraints

Grid Capacity (per slot)

Sum of power across all vehicles in slot k must not exceed the available transformer headroom, after subtracting building base load and ML-reserved capacity for predicted future arrivals:

$$\sum_i x_{i,k} \leq P_{\max} - \text{base_load}_k - \text{reserved}_k \text{ for all } k$$

Hardware Limit (per vehicle per slot)

Power assigned to vehicle i in slot k is bounded by the physical charger rating and zero if the vehicle is not present:

$$0 \leq x_{i,k} \leq p_i^{\max} \times \text{presence}_{i,k}$$

Energy Cap (per vehicle)

The total energy delivered to a vehicle cannot exceed what it needs:

$$\sum_k x_{i,k} \cdot \Delta t \leq e_i \text{ for all } i$$

Minimum Guarantee (per vehicle)

Every vehicle is guaranteed at least 20% of its requested energy, capped at the physical maximum achievable within its stay window. This prevents complete starvation:

$$\sum_k x_{i,k} \cdot \Delta t \geq 0.20 \cdot e_i \text{ for all } i$$

ML-driven capacity reservation: The demand forecast (M2) outputs `predicted_count` and `predicted_kwh` for each future time slot. The optimizer computes `reserved_k = predicted_count × avg_charge_kw` and subtracts this from the available headroom. This ensures that when predicted vehicles actually arrive, there is grid capacity waiting for them.

5. Objective Function (2-Phase LP)

The optimizer solves two LP problems sequentially. Phase 1 finds the maximum total energy achievable; Phase 2 uses that as a floor constraint and optimizes fairness.

Phase 1 — Maximize Total Energy Delivered

Each vehicle $\times$ slot variable is weighted by an **urgency score** (vehicles with less remaining charging time get higher weight) and a **front-loading decay** (earlier slots in a vehicle's window are preferred over later slots):

$$\begin{aligned} &\text{maximize } \sum_i \sum_k w_i \cdot \text{decay}_{i,k} \cdot x_{i,k} \cdot \Delta t \text{ where } w_i = 1.0 + 0.5 \times \\ &\min(\text{urgency}_i, 2.0) \text{ decay}_{i,k} \in [0.3, 1.0] \text{ (1.0 at arrival, 0.3 at departure)} \\ &\text{urgency}_i = e_i / (\text{available_slots} \times \Delta t \times p_i^{\max}) \end{aligned}$$

Why front-loading? ML predicted a vehicle stays 8 hours, but it might leave after 3. By giving higher weight to early slots, the optimizer delivers energy quickly — making the schedule **robust to departure uncertainty** even when the ML prediction is imperfect.

Phase 2 — Max-Min Fairness

Given the optimal total energy E^* from Phase 1, Phase 2 introduces a scalar variable t representing the minimum satisfaction fraction across all vehicles, and maximises it:

```
maximize t subject to:  $\text{sat}_i = \sum_k x_{i,k} \cdot \Delta t / e_i \geq t$  for all  $i \in \{1, \dots, N\}$   
 $x_{i,k} \cdot \Delta t \geq 0.95 \cdot E^*$  (efficiency floor)  $t \leq 1.0$ 
```

This eliminates the 100%/0% extreme outcomes. A vehicle that physically cannot reach 10% satisfaction within its time window (e.g. very short stay + large battery) is excluded from the fairness constraint so it does not drag the minimum down for the entire fleet.

6. Goal Summary

Primary Goal: Maximize the total energy delivered to all connected EVs within physical grid capacity, while ensuring no vehicle is starved (20% minimum guarantee), preferring early delivery to handle departure uncertainty from ML predictions, and reserving capacity for ML-predicted future arrivals — all subject to transformer, hardware, and energy constraints.

7. ML Value — Quantified Comparison

The system automatically compares all three strategies on every run to prove that ML integration improves outcomes over the no-ML baseline:

Departure estimate	ML q50 prediction	ACN per-hour mean	ACN per-hour mean
Capacity reservation	ML demand forecast	None	None
Departure MAE	~112 min	~184 min	~184 min
Scheduling method	2-phase LP	2-phase LP	Greedy FCFS
Min. guarantee	20% per vehicle	20% per vehicle	None
Fairness objective	Max-min satisfaction	Max-min satisfaction	First-come only

Table 5 — Side-by-side strategy comparison.

The ML departure predictor achieves ~39% lower MAE than the historical mean baseline. This translates directly to the optimizer receiving more accurate `departure_slot` values → tighter, more efficient power allocation → higher overall satisfaction percentage and more energy delivered per session.

8. Online Feedback & Retraining Loop

After each simulation, the gateway writes two feedback files to disk:

File	Content	Used for
collected_sessions.csv	Actual session records (arrival, departure, kWh)	Periodic retraining of M1 departure model

File	Content	Used for
departure_feedback.csv	Predicted vs. actual stay duration per vehicle	Drift monitoring (drift_monitor.py) + MAE tracking

Table 6 — Feedback files written after each simulation run.

A Kubernetes CronJob (retrain-cronjob.yaml) re-runs the training pipeline weekly using accumulated real-session data, enabling the ML models to improve over time as more charging behaviour is observed.